

Gručenje v hadronske curke

1 Kaj so hadronski curki

Pri visoko-energijskih trkih med osnovnimi delci nastane veliko število delcev v končnem stanju, katerih fizikalne lastnosti, kot je energija, se lahko zaznajo s posebnimi detektorji. Med temi delci so pogosto prisotni hadroni, kot so proton, nevtron ali pion, ki jih pogosto zaznamo tudi v kozmičnih žarkih. Več primerov hadronov je na voljo na spletni strani PDG¹, v sekcijah *Mesons* (mezoni - hadroni s celoštevilskim spinom) in *Baryons* (barioni - hadroni s polcelim spinom).

Danes je znano, da hadroni niso osnovni delci, temveč so sestavljeni iz kvarkov, ki jih veže močna interakcija. Standardni Model² opisuje spekter osnovnih delcev in dinamiko med njimi, ne pa tudi eksplicitno vezanih stanj. To vodi do pomembnega vprašanja: kako povezati kvarkovska končna stanja s hadroni, ki jih zaznamo v eksperimentih? Prostih kvarkov namreč ni mogoče zaznati, saj se zaradi narave močne interakcije hitro vežejo v hadrone.

Poleg tega močna interakcija povzroči tudi sevanje gluonov - prenašalcev močne interakcije - iz kvarkov. Ti gluoni nato lahko tvorijo nove kvarke ali pa se vežejo v hadrone. Ta proces se lahko večkrat ponovi, kar v končnem stanju privede do velikega števila hadronov (hadronizacija), ki izvirajo iz nekaj kvarkov nastalih v osnovni interakciji, glej sliko 1. Hadroni so pogosto zelo kolimirani, kar pomeni, da je mogoče ločiti skupine hadronov, ki izvirajo iz enega kvarka, od drugih. Te skupine hadronov imenujemo hadronski curki (*hadronic jets*).

Transformacija od kvarkov do končnih hadronskih curkov ne spremeni signifikantno informacij o energiji in gibalni količini, kar pomeni, da curke lahko uporabimo za študijo procesov v fiziki osnovnih delcev ter primerjavo s teorijo, četudi ta govori v jeziku kvarkov [1, 2].

V primeru, ko v nekem procesu pričakujemo dva kvarka, se v eksperimentu načeloma pričakuje dva hadronska curka. Vendar pa bo v praksi število hadronskih curkov običajno večje, saj lahko tudi začetna stanja trkov, kot v primeru trkalnika LHC, sevajo gluone. S tem se celoten proces hadronizacije lahko ponovi tudi za te gluone, kar povzroči nastanek dodatnih hadronov v končnem stanju. To težavo običajno rešimo z uporabo filtriranja curkov na podlagi kinematičnih lastnosti, s ciljem obravnavati samo najbolj visokoenergijske curke, ki so najverjetneje povezani z osnovnim procesom.

Za študij kinematičnih lastnosti curkov se v visoko-energijskih trkalnikih osnovnih delcev pogosto uporablja koordinatni sistem, ki poenostavi analizo. Poljuben delec z maso m v končnem stanju je mogoče opisati s četvercem gibalne količine $p = (E, \mathbf{p})$, kjer je $E = \sqrt{m^2 + |\mathbf{p}|^2}$ energija delca, $\mathbf{p}$ pa vektor gibalne količine. V visokih energijah so mase delcev pogosto zanemarljive, zato se namesto koordinat (p_x, p_y, p_z) uporabljajo koordinate (p_T, η, ϕ) , kjer so:

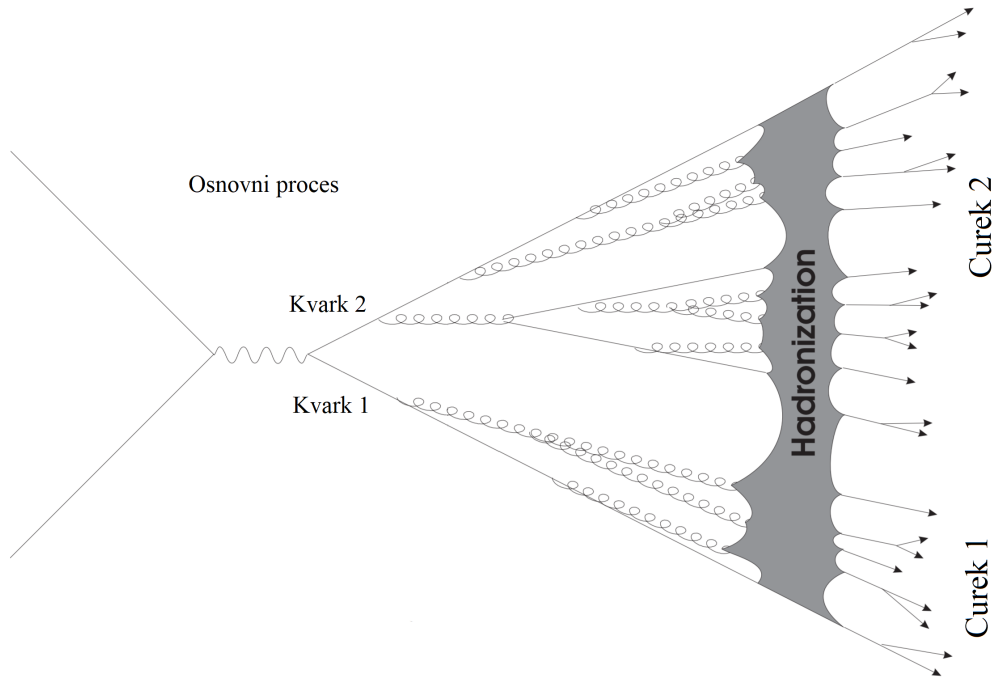
- p_T transverzalna gibalna količina, ki je pravokotna na smer curka,
- η pseudorapidnost, ki opisuje hitrost delca neodvisno od njegove mase,
- ϕ azimutni kot.

2 Algoritmi gručenja

Identifikacija delcev, ki pripadajo istemu kolimiranemu hadronskemu curku, poteka z uporabo algoritmov gručenja v ravnini kotnih spremenljivk (η, ϕ) . Pri tem se predvideva, da se hadroni, ki izvirajo iz skupnega osnovnega kvarka, v tej ravnini nahajajo bližje skupaj.

¹Particle Data Group - <https://pdglive.lbl.gov/Viewer.action>

²Teorija osnovnih delcev, katere del je tudi kvantna kromodinamika (QCD - *quantum chromodynamics*), ki opisuje močno interakcijo.



Slika 1: Skica poteka hadronizacije iz dveh kvarkov v končnem stanju osnovnega procesa. Prirejeno iz [3].

Algoritem K -means Algoritem K -means je eden najpreprostejših algoritmov gručenja. Njegova naloga je razdeliti množico s število točk N na K gruč. Vsaki gruči $j = 1, \dots, K$ pripada podmnožica točk in povprečje točk posamezne gruče μ_j so znana tudi kpt centriodi. Algoritem določa centriode tako, da minimizira razdaljo vsake x_i , kjer $i = 1, \dots, N$ točke do pripadajočega centroida μ_j , kjer $j = 1, \dots, K$:

$$\sum_{i=1}^N ||x_i - \mu_j||^2 = \min . \quad (1)$$

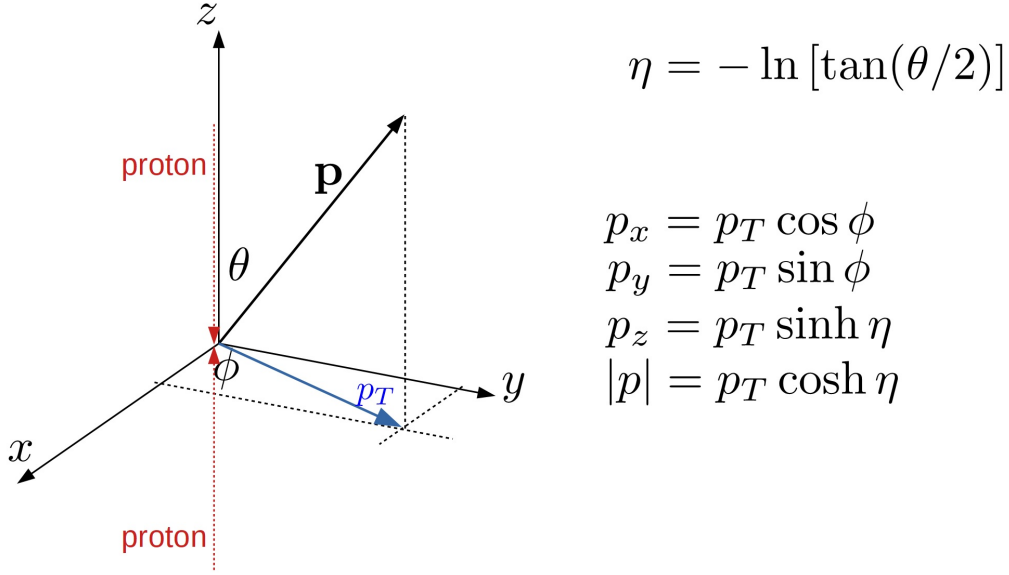
V osnovi algoritem K -means vključuje tri ključne korake. V prvem koraku se določijo začetne vrednosti k centroidov. Te lahko izberemo z naključnim vzorčenjem znotraj ustreznih intervalov, skladnih s podatki, ali pa preprosto izberemo k naključnih točk iz množice podatkov. Nato se izvede naslednja dva koraka:

1. Na podlagi trenutnih vrednosti centroidov vsaki točki $i = 1, \dots, N$ pripišemo gručo, katere centroid je najbližji (privzemi Evklidsko metriko).
2. Posodobimo vrednosti centroidov tako, da za vsako gručo j izračunamo povprečje μ_j vseh točk, ki ji pripadajo.

Ta dva koraka ponavljamo, dokler algoritem ne konvergira, tj. dokler se vrednosti centroidov med iteracijami bistveno ne prenehajo spreminjati.

Pridobljene gruče predstavljajo curke delcev, pri čemer so vrednosti (η, ϕ) posameznega curka določene z vrednostmi pripadajočega centroida, skupna transversalna gibalna količina gruče p_T pa je izračunana kot vsota transversalnih gibalnih količin delcev, ki pripadajo tej gruči.

Algoritem K -means ima kljub svoji preprostosti več pomembnih slabosti. Pogosto konvergira v lokalne minimume, je izjemno občutljiv na začetno izbiro centroidov in predpostavlja, da so gruče sferične, kar lahko vodi do neustreznih rezultatov pri bolj kompleksnih podatkih. Za učinkovito uporabo tega algoritma pri gručenju curkov hadronov pa bi moral zadostiti tudi naslednjima dvema lastnostma:



Slika 2: Skica koordinatnega sistema, uporabljenega v fiziki trkalnikov osnovnih delcev in transformacija med koordinatnima sistemoma.

1. **Infrardeča varnost:** Rezultati gručenja ne smejo biti občutljivi na dodajanje novih delcev z majhno transversalno gibalno količino.
2. **Kolinearna varnost:** Rezultati gručenja ne smejo biti odvisni od zamenjave delca z gibalno količino p_i z dvema vzporednima (kolinearnima) delcema j in k , pri čemer velja $p_i = p_j + p_k$.

V praksi se za identifikacijo hadronskih curkov uporabljajo algoritmi hierarhičnega gručenja.

Hierarhično gručenje V nadaljevanju so predstavljeni trije algoritmi, ki se razlikujejo po vrednosti parametra p , ki nastopa zgolj v eksponentu.

Seznam delcev, ki jih izmerimo, imenujemo seznam *protocurkov*. Seznam, ki ga dobimo po gručenju, imenujemo seznam *pravih curkov*. Na začetku je ta seznam prazen in se polni med potekom algoritma. Algoritem vključuje prost parameter $R \sim 1$, ki je določen glede na geometrijo detektorja. Postopek gručenja je opisan v naslednjih korakih:

1. Za vsak protocurek i izračunamo količino $d_i = p_{T,i}^{2p}$ ter za vsak par protocurkov (i, j) izračunamo razdaljo:

$$d_{ij} = \min(p_{T,i}^{2p}, p_{T,j}^{2p}) \frac{(\eta_i - \eta_j)^2 + (\phi_i - \phi_j)^2}{R^2}. \quad (2)$$

Pri tem glej vrednost za $(\phi_i - \phi_j)^2$ kot kvadrat kota med dvema delcema na krožnici. Skratka, če je $\phi_i > \phi_j$ potem je absolutna vrednost kota med delcema $\min(\phi_i - \phi_j, \phi_j + 2\pi - \phi_i)$. Količina ϕ živi na mnogoterosti krožnica S^1 in operacija razlike "−" definirana na krožnici je drugačna kot na premici ($\mathbb{R}$).

2. Poiščemo minimalno razdaljo $d_{\min}$ med množicama $\{d_i; i = 1, \dots, N\}$ in $\{d_{ij}; i, j = 1, \dots, N\}$.

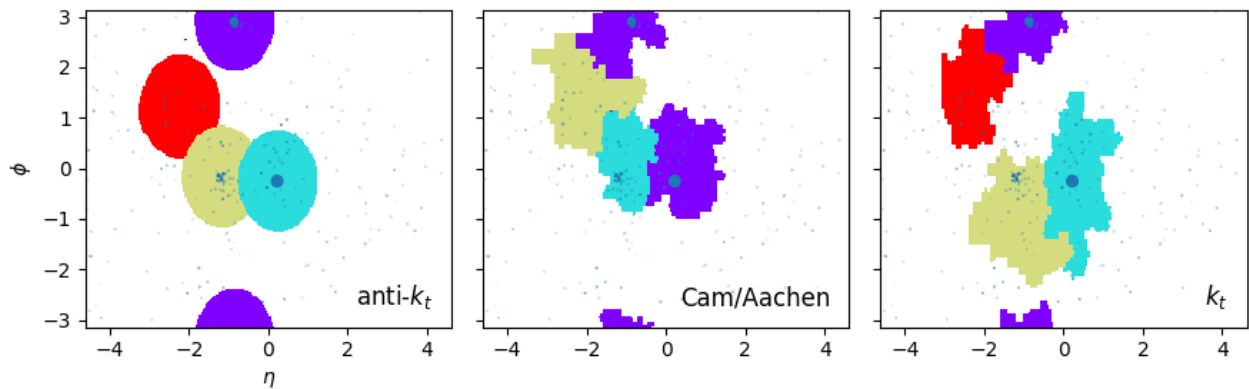
3. Če je minimalna razdalja d_{ij} , združimo protocurka i in j v nov protocurek k , pri čemer določimo:

$$\begin{aligned} p_{T,k} &= p_{T,i} + p_{T,j}, \\ \eta_k &= \frac{p_{T,i}\eta_i + p_{T,j}\eta_j}{p_{T,k}}, \\ \phi_k &= \frac{p_{T,i}\phi_i + p_{T,j}\phi_j}{p_{T,k}}. \end{aligned} \quad (3)$$

4. Če je minimalna razdalja d_i , protocurek i odstranimo iz seznama protocurkov in ga vpišemo v seznam pravih curkov.

5. Postopek se vrne na 1. korak, dokler seznam protocurkov ni prazen.

Z izbiro parametra $p \in \{-1, 0, 1\}$ (ki nastopa zgolj v eksponentu) reproduciramo znane algoritme: *anti- k_t* , Cambridge-Aachen in k_t [4–6]. Primer gručenja z vsemi tremi algoritmi je prikazan na sliki 3.



Slika 3: Primer gručenja v hadronske curke z algoritmi *anti- k_t* , Cambridge-Aachen in k_t . Delci iz dogodka so prikazani z modrimi točkami, kjer večja točka pomeni večji p_T . Obarvana področja predstavljajo curke s $p_T > 10$ GeV. Vzeto iz <https://github.com/scikit-hep/pyjet/blob/master/examples>.

3 Naloga

1. Implementiraj algoritem K -means po postopku, opisanem v poglavju 2 ali predstavljenem na predavanjih. Preizkusi implementacijo na priloženih podatkih v datoteki `gauss.npy`. Podatki so podani kot polje velikosti 400×2 , ki predstavlja 400 delcev, pri čemer ima vsak delec dva parametra. Parametra sta naključno vzorčena iz dveh Gaussovih porazdelitev. Iz najdenih gruč določi povprečne vrednosti in standardne odklone obeh Gaussovih porazdelitev. Primerjaj rezultate z metodo Gaussian Mixture Model (GMM) iz knjižnice `sklearn`.
2. V datoteki `h_bb_sorted.npy` je shranjeno polje dogodkov, pri čemer vsak dogodek predstavlja trk dveh protonov, ob katerem nastane Higgsov bozon, ki razpade na dva kvarka b in $\bar{b}$. Vsak dogodek vsebuje različno število delcev, katerih parametri so podani v obliki Pythonovega tupla (p_T, η, ϕ, m) . Masa m v tej nalogi ni pomembna. Vrednosti so simulirane z upoštevanjem efekta hadronizacije. **Upoštevaj, da je geometrija cilindrična in da gručenje poteka na mnogoterosti $\mathbb{R} \times S^1$, torej v ravnini (η, ϕ) s periodičnimi robnimi pogoji za ϕ .**

Uporabi implementiran algoritem K -means na prvem dogodku iz datoteke `h_bb_sorted.npy` za različno število centroidov K . Poišči gruči z največjo transversalno gibalno količino p_T , ki

pripadata curkama kvarkov b in $\bar{b}$. Določi četverca gibalnih količin p_1 in p_2 obeh curkov ter s tem izračunaj maso Higgsovega bozona:

$$m_h = \sqrt{(p_1 + p_2)_\mu (p_1 + p_2)^\mu} = \sqrt{(E_1 + E_2)^2 - (\mathbf{p}_1 + \mathbf{p}_2)^2}. \quad (4)$$

Pri tem uporabi približek $E_{1,2} \approx |\mathbf{p}_{1,2}|$. Do podatkov v datoteki dostopaš z ukazom: `data = np.load('h_bb_sorted.npy', allow_pickle=True)`. Izmerjena masa Higgsovega bozona je 125 GeV.

- Preveri, ali implementiran algoritem K -means zadošča infrardeči varnosti, če je uporabljen na prvem dogodku iz datoteke `h_bb_sorted.npy`. To pomeni, da postopoma odstranjuješ delce z najmanjšo transversalno gibalno količino p_T in spremljaš, kako se rekonstruirana masa Higgsovega bozona spreminja s številom delcev v dogodku.
- Implementiraj algoritem hierarhičnega gručenja in ga preizkusi na prvem dogodku. Izračunaj maso Higgsovega bozona za vse tri vrednosti parametra $p \in \{-1, 0, 1\}$ in za $R = 6$. Kaj se zgodi, če povečaš ali zmanjšaš R ? (Parameter R sledi lastnostim detektorja oziroma umeritvi metode na dani detektor; v praksi je fiksni.)

Implementiran algoritem hierarhičnega gručenja bo mogoče deloval zelo počasi. V praksi se za gručenje hadronskih curkov uporablja orodje `fastjet` [7, 8], v katerem so algoritmi s $p \in \{-1, 0, 1\}$ optimizirani in implementirani v C++. Na voljo je tudi knjižnica za Python, `pyjet`³, ki omogoča enostavno uporabo optimiziranih algoritmov neposredno iz Python skript. Za uporabo algoritmov je potrebno najprej pretvoriti podatke z naslednjim ukazom:

```
from pyjet import DTYPE_PTEPM, cluster
import numpy as np

data = np.load('h_bb_sorted.npy', allow_pickle = True)
event = np.array(data[0], dtype=DTYPE_PTEPM) # prvi dogodek
```

Podrobnosti so opisane v dokumentaciji knjižnice⁴. Algoritem k_t lahko poženemo z naslednjima ukazoma ($p = 1$ za k_t algoritem):

```
sequence = cluster(event, ep=False, R=0.6, p=1)
incl_jets = sequence.inclusive_jets()
```

Objekt `incl_jets` predstavlja seznam inkluzivnih curkov, kar pomeni, da so v obravnavi zajeti vsi delci iz dogodka. Curki so podani v obliki objekta `PseudoJet`, s katerim lahko dostopamo, na primer, do energije j -tega curka z ukazom `incl_jets[j].e`, do ene od komponent gibalne količine pa z ukazom `incl_jets[j].px`.

Za naslednje naloge uporabi algoritme iz knjižnice `pyjet`, podatke iz datoteke `h_bb_sorted.npy` ter postopek izračuna rekonstruirane mase Higgsovega bozona, kot je opisan v nalogi 2 (2. odstavek).

- Izvedi test infrardeče varnosti za implementirani algoritem in algoritem iz knjižnice `pyjet` ter ju primerjaj (zadostuje za $p = 1$). Obenem izmeri čas izvajanja obeh algoritmov in primerjaj odvisnost časa izvajanja od števila delcev v logaritemski skali. **5. naloga je primerjava infrardeče varnosti, medtem ko je 6. naloga primerjava časa izvajanja.**
- Poženi algoritem iz knjižnice `pyjet` na vseh dogodkih, podanih v datoteki `h_bb.npy`, in nariši spekter rekonstruiranih invariantnih mas Higgsovega bozona. Razmisli, kako bi določil maso Higgsovega bozona skupaj s pripadajočo negotovostjo, ter ju izračunaj (zadostuje $p = 1$).

Če želiš vizualizirati gručenje curkov, kot je to prikazano na sliki 3, si lahko pomagaš s kodo, ki je na voljo na GitHub strani paketa `pyjet` (https://github.com/scikit-hep/pyjet/blob/master/examples/plot_jet_areas.py).

³<https://github.com/scikit-hep/pyjet>

⁴<https://github.com/scikit-hep/pyjet>

Viri

- [1] M. H. Seymour, Jets in QCD, AIP Conf. Proc. 357 (1996) 568–587. [arXiv:hep-ph/9506421](#), [doi:10.1063/1.49625](#).
- [2] A. Banfi, Introduction, in: Hadronic Jets, 2053-2571, Morgan and Claypool Publishers, 2016, pp. 1–1 to 1–12. [doi:10.1088/978-1-6817-4073-7ch1](#).
URL <http://dx.doi.org/10.1088/978-1-6817-4073-7ch1>
- [3] D. Buskulic, D. Casper, I. Bonis, D. Decamp, P. Ghez, C. Goy, J. Lees, A. Lucotte, M. Minard, P. Odier, B. Pietrzyk, F. Ariztizabal, M. Chmeissani, J. Crespo, I. Efthymiopoulos, E. Fernandez, M. Fernandez-Bosman, V. Alcalde, L. Garrido, G. Zobernig, Measurement of α_s from scaling violations in fragmentation functions in e^+e^- annihilation (01 1995).
- [4] S. D. Ellis, D. E. Soper, Successive combination jet algorithm for hadron collisions, Phys. Rev. D 48 (1993) 3160–3166. [arXiv:hep-ph/9305266](#), [doi:10.1103/PhysRevD.48.3160](#).
- [5] Y. L. Dokshitzer, G. Leder, S. Moretti, B. Webber, Better jet clustering algorithms, JHEP 08 (1997) 001. [arXiv:hep-ph/9707323](#), [doi:10.1088/1126-6708/1997/08/001](#).
- [6] M. Cacciari, G. P. Salam, G. Soyez, The anti- k_t jet clustering algorithm, JHEP 04 (2008) 063. [arXiv:0802.1189](#), [doi:10.1088/1126-6708/2008/04/063](#).
- [7] M. Cacciari, G. P. Salam, G. Soyez, FastJet User Manual, Eur. Phys. J. C 72 (2012) 1896. [arXiv:1111.6097](#), [doi:10.1140/epjc/s10052-012-1896-2](#).
- [8] M. Cacciari, G. P. Salam, Dispelling the N^3 myth for the k_t jet-finder, Phys. Lett. B 641 (2006) 57–61. [arXiv:hep-ph/0512210](#), [doi:10.1016/j.physletb.2006.08.037](#).